Procesamiento de imágenes: la imagen digital

Matriz de píxeles, resolución espacial, profundidad de bits, canales y tipos de imagen

Alumno	Carlos Fernando Serrano Amador	**Fecha**	13 de noviembre de 2026
Materia	Visión Artificial	**Modalidad**	Individual · práctica de laboratorio
Entregable	Tres scripts (`p1_paso1`, `p1_paso2`, `p1_paso3`) y este reporte con capturas y respuestas.		

Resumen. Programé tres scripts incrementales con OpenCV, NumPy y Matplotlib. El primero carga una imagen, la muestra, lee y modifica píxeles y la recorre como matriz. El segundo aplica muestreo (resolución) y cuantización (bits por canal). El tercero separa canales, calcula el gris de cuatro maneras y construye una máscara binaria. Para las capturas usé una foto que tomé con mi webcam (`recursos/webcam.png`).

1. Objetivo y entorno

El objetivo fue comprender que una imagen digital es una matriz de números con una resolución, una profundidad de bits y uno o varios canales, y comprobarlo escribiendo los scripts que la cargan, la transforman y la muestran. Creé el entorno con `python -m venv va` y `pip install -r requirements.txt`:

Componente	Versión	Uso en la práctica
Python	3.14.8	Intérprete y entorno virtual `va`
OpenCV (`cv2`)	5.0.0	Lectura, color, resize, umbral, ventanas
NumPy	2.5.4	Arreglos, indexado, memoria, histogramas
Matplotlib	3.11.2	Figuras con varios paneles y anotaciones

2. Paso 1 · Cargar, mostrar e inspeccionar

En la sección `[BASE]` concentré `cargar_imagen`, `capturar_webcam`, `obtener_imagen`, `imprimir_info` y `a_rgb`. En el programa principal imprimí las propiedades, leí el píxel `(y=100, x=200)`, pinté un cuadro rojo de 40×40 sobre una copia y recorté una región de 6×6 sobre el logotipo de la sudadera, en `(y=420, x=330)`. La región por defecto caía en la pared, así que la moví a un borde con más información.

```
color      shape=(480, 640, 3)   ancho=640  alto=480  canales=3 dtype=uint8 min= 54 max=196 memoria=900 KB
gris       shape=(480, 640)      ancho=640  alto=480  canales=1 dtype=uint8 min= 65 max=185 memoria=300 KB
color -> B=159 G=156 R=150    gris -> 155
merge([b, g, r]) es idéntica a la original: True
```

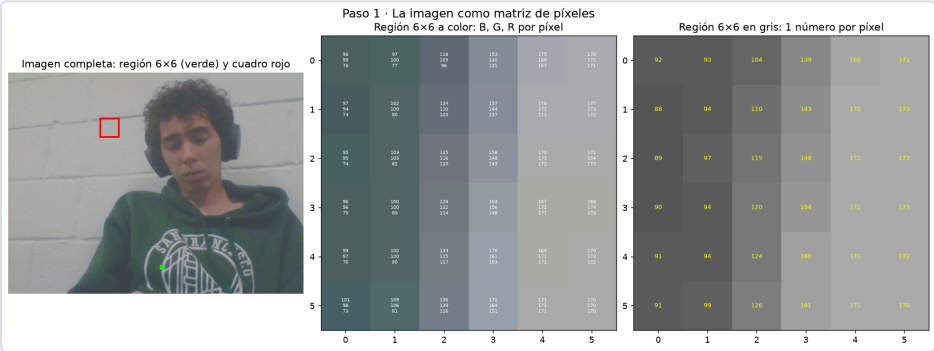


Figura 1. La foto de mi webcam con la región 6×6 sobre el logotipo (verde) y el cuadro rojo; a la derecha, los valores B, G, R de cada píxel y el valor único de la imagen en gris.

Preguntas del paso 1

1. **¿Qué significa cada número de `shape = (480, 640, 3)`?** 480 filas (alto), 640 columnas (ancho) y 3 canales (B, G, R). Guarda 480·640·3 = **921 600 números**; como cada uno es `uint8`, ocupa 921 600 bytes = 900 KB.
2. **¿Por qué el color ocupa el triple que el gris?** Porque el gris tiene un solo canal y el color tres, con el mismo tipo de dato y el mismo número de píxeles: 3 × 300 KB = 900 KB.
3. **El cuadro empieza en `y=100, x=200`: ¿a 100 px de qué borde?** A 100 px del borde **superior** (y crece hacia abajo) y a 200 px del borde **izquierdo** (x crece hacia la derecha).
4. **¿Qué pasa si `marcada = imagen` sin `.copy()`?** Ambas variables apuntan al mismo arreglo: pintar el cuadro modifica también la "original" y la ventana "Original" mostraría el cuadro rojo. No se duplica memoria; el cambio es un alias, no una copia.
5. **En la región 6×6, ¿cuál es más claro en gris, el blanco del logotipo o el verde de la sudadera?** El blanco. El verde tiene B=96, G=99, R=76 y un gris de 92; el blanco tiene B=178, G=171, R=172 y un gris de 172. La fórmula `0.299R + 0.587G + 0.114B` da un valor alto cuando los tres canales son altos; el verde solo destaca en G y pierde en R y B.

3. Paso 2 · Resolución espacial y profundidad de bits

Conservé la `[BASE]` y agregué `reducir_resolucion` (muestreo con `INTER_AREA` y ampliación con `INTER_NEAREST`) y `reducir_bits` (cuantización: `niveles = 2**bits`, `paso = 256 // niveles`). Medí la memoria con `nbytes` y calculé la teórica como `píxeles × bits / 8`.

factor	resolución	píxeles	memoria
1	640×480	307 200	900.0 KB
2	320×240	76 800	225.0 KB
4	160×120	19 200	56.2 KB
8	80×60	4 800	14.1 KB
16	40×30	1 200	3.5 KB
32	20×15	300	0.9 KB

bits	niveles	distintos	teórica	real
8	256	121	300 KB	300 KB
4	16	8	150 KB	300 KB
3	8	4	112 KB	300 KB
2	4	2	75 KB	300 KB
1	2	2	38 KB	300 KB

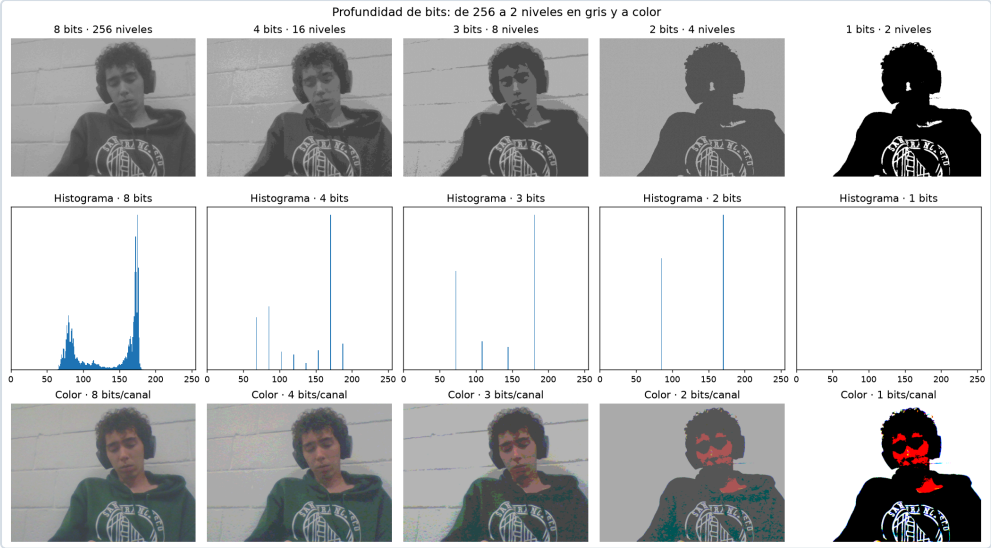


Figura 2. Profundidad de bits: la misma foto con 8, 4, 3, 2 y 1 bit por canal. La segunda fila es el histograma en gris y la tercera el color; los falsos contornos aparecen en la pared y en la piel.

Preguntas del paso 2

1. **¿A partir de qué resolución ya no reconoces el rostro? ¿Y el logotipo?** El rostro se reconoce hasta 40×30; en 20×15 ya es una mancha. El logotipo “San Francisco” se lee hasta 160×120 y a 80×60 solo quedan manchas; en 40×30 desaparece. El detalle fino exige más píxeles que la silueta.
2. **¿Cuántas veces menos memoria al dividir ancho y alto entre 2?** Cuatro veces, no dos: el número de píxeles es ancho×alto, y al reducir cada dimensión a la mitad el área queda (1/2)·(1/2) = 1/4. La tabla lo confirma: 900 → 225 → 56.2 KB.
3. **¿Con cuántos bits aparecen los falsos contornos y dónde primero?** Con 4 bits se insinúan y con 3 o 2 bits son evidentes. Aparecen primero en la pared y en la piel, porque son zonas de cambio suave: la diferencia entre píxeles vecinos es menor que el escalón de cuantización. El histograma se reduce a 8, 4 o 2 barras y el degradado se convierte en franjas planas.
4. **La memoria real no baja al reducir bits. ¿Por qué?** Porque `reducir_bits` sigue guardando cada píxel en un `uint8` (8 bits), aunque solo use 2, 4 u 8 niveles. Para que bajara habría que empaquetar los bits (`np.packbits`) o usar un tipo de 1/2/4 bits y descomprimir al mostrar; el archivo PNG también aprovecha la repetición y comprime.
5. **¿Qué afecta más: 640×480→160×120 (16× menos píxeles) o 8→2 bits (4× menos bits)?** En mi imagen, la cuantización a 2 bits: solo quedan 4 niveles y la piel y la pared se llenan de franjas, aunque la silueta se conserva. A 160×120 el rostro y el logotipo todavía se reconocen. La resolución se vuelve más destructiva cuando se baja a 40×30 o 20×15, donde ya no se identifica el objeto.

4. Paso 3 · Tipos de imagen y canales de color

Separé los canales con `cv2.split`, reconstruí la imagen con `cv2.merge` (idéntica, `True`) y calculé cuatro versiones del gris: `cvtColor`, la fórmula ponderada, el promedio en `uint16` y el promedio en `uint8` (a propósito, con desbordamiento). La diferencia máxima entre `cvtColor` y la fórmula fue **1** nivel, por el redondeo. En el píxel `(60, 250)` la suma real B+G+R es 520 y en `uint8` da 8: 520 − 512 = 8 (módulo 256).



Figura 3. La foto y sus tres canales: arriba cada canal en gris; abajo el error de mostrar BGR como RGB y cada canal en su color. El canal G es el que mejor representa la sudadera verde; el logotipo blanco aparece alto en los tres canales.

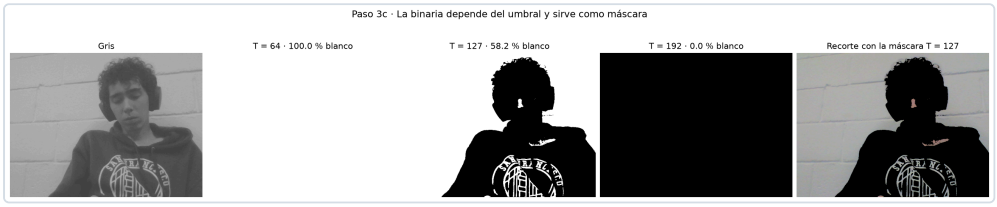


Figura 4. La binaria depende del umbral: con T=64 el 100 % queda blanco, con T=127 el 58.2 % y con T=192 el 0 %. A la derecha, el recorte que conserva solo los píxeles blancos de la máscara T=127.

Preguntas del paso 3

1. **Mi sudadera es verde: ¿en qué canal se ve más clara y en cuál más oscura?** En G se ve más clara (≈99) y en R más oscura (≈76); B queda en ≈96. El verde refleja sobre todo verde, por eso el canal G es el más alto; el blanco del logotipo, en cambio, tiene los tres canales altos (≈170) y se ve claro en todos.
2. **¿Por qué al mostrar BGR sin convertir la sudadera se ve azulada y la piel azulada?** Porque Matplotlib interpreta el primer canal como R, el segundo como G y el tercero como B; al leer BGR como RGB se intercambian rojo y azul. La sudadera verde y la piel cálida pierden el rojo y se ven con tinte azul.
3. **¿Por qué `promedio_mal` tiene manchas oscuras en las zonas claras?** Porque `b + g + r` se hace en `uint8`: al superar 255 el resultado da la vuelta (módulo 256). La pared y la piel tienen sumas mayores que 255 y producen valores pequeños, que se ven negros; es el desbordamiento que queríamos mostrar.

Preguntas del paso 3 (continuación)

4. **¿Qué umbral separa mejor a la persona del fondo? ¿Hay uno perfecto?** T=127 separa la silueta de la persona (58.2 % de blancos). Con T=64 todo queda blanco porque el gris mínimo es 65; con T=192 todo queda negro porque el máximo es 185. No existe un umbral perfecto: la pared clara, la piel y el logotipo comparten niveles altos, y la sudadera se confunde con el pelo y las sombras. El mejor corte está entre 120 y 140, pero siempre deja parte de la pared o del cuerpo fuera.
5. **La binaria ocupa lo mismo que el gris aunque solo tenga 2 valores. ¿Cuántos bits bastarían?** Un bit por píxel: $2^1 = 2$ valores. Para 640×480 serían 307 200 bits = 38 400 bytes = 37.5 KB (1/8 de los 300 KB), más una pequeña cabecera; empaquetando con `np.packbits` se lograría.

5. Conclusiones

Con esta práctica comprobé que una imagen de OpenCV es un arreglo de NumPy y que casi todo lo que se hace en visión artificial se reduce a leer y escribir índices de esa matriz. La resolución espacial y la profundidad de bits son dos pérdidas distintas: el muestreo borra detalle y objetos pequeños; la cuantización crea falsos contornos. El color son tres matrices que puedo separar, recombinar y ponderar; el gris no es el promedio simple, sino una suma con pesos que reflejan la sensibilidad del ojo. La imagen binaria es la máscara que usaré en las prácticas 2 y 3.

> **Nota.** La foto se tomó con mi webcam y se guardó como `recursos/webcam.png`; el script también acepta `--camara`: con ESPACIO se captura una foto de la webcam y se usa como imagen de trabajo.

Referencias. Guía P01 del curso; documentación de OpenCV 5.0 (`imread`, `cvtColor`, `resize`, `threshold`, `split`); documentación de NumPy (`ndarray`, `nbytes`, `unique`).